



Ottimizzazione 5 - Branch

Principi della Programmazione Parallela CdL Informatica - A.A. 2025/2026



Università
di Catania



INAF
ISTITUTO NAZIONALE
DI ASTROFISICA



Ottimizzazione

Outline



Intro



Cache &
Memoria



Branch



Cicli



Branch

Outline

- a) Definizione di branch condizionali
 - b) Flusso di esecuzione dipendente dai dati
 - c) Flusso di dati dipendente dai dati
 - d) Impatto dei branch condizionali sull'efficienza del codice
 - e) 4 esempi su come pulire/ristrutturare un codice
 - 1. branch condizionali all'interno dei cicli
 - 2. flussi di dati imprevedibili
 - 3. ordinamento di due array
 - 4. riempimento di una matrice
-



Branch

Non perdere il controllo

Ogni volta che (i) la sequenza di operazioni che devono essere eseguite o (ii) la sequenza di dati da elaborare dipendono da una condizione, ad esempio dall'esito di un test eseguito su alcuni dati o risultati, abbiamo un'esecuzione *condizionale*.

L'architettura moderna offre 2 distinte istruzioni di basso livello per implementare un'esecuzione condizionale su un test:

- modifica del *flusso di controllo* → **flusso di esecuzione dipendente dai dati**
- modificando il *flusso di dati* → **flusso di dati dipendente dai dati**



Branch

Controllo di basso livello

Vediamo prima il flusso di esecuzione condizionale.

A livello macchina, il modo per modificare il flusso di esecuzione è attraverso un'istruzione **di salto (jump)**, che fa sì che il controllo venga passato a una sezione di codice diversa.

L'istruzione di salto può essere *condizionale*, quando la sua esecuzione dipende dall'esito di un'operazione (un test), oppure *incondizionata* se non lo è.

Una chiamata di funzione è un'istruzione di salto di un tipo particolare, che qui non ci interessa.



Branch

Controllo di basso livello

jmp è l'unica istruzione *incondizionata* ; accetta una destinazione *diretta* (specificata da un'etichetta) o una destinazione *indiretta* (specificata da un indirizzo in un registro o nella memoria).

je , jne e le altre sono istruzioni *condizionali* : cioè la loro esecuzione dipende da una condizione.

Queste istruzioni accedono ai valori memorizzati nei bit del registro flag, un registro speciale in cui la CPU iscrive alcuni risultati caratteristici dell'ultima operazione aritmetica o logica

CF	CARRY FLAG ; è stato generato un carry out dell'msb, segnalando un overflow nell'operazione non firmata
ZF	ZERO FLAG ; l'operazione più recente ha prodotto un risultato pari a zero
SF	SIGN FLAG ; l'operazione più recente si è conclusa con un risultato
OF	FLAG DI OVERFLOW ; un overflow in complemento a due, negativo o positivo.

Questa tabella mostra alcune delle istruzioni di salto di basso livello normalmente disponibili sulle CPU moderne.

jmp	<i>Etichetta</i> <i>*Operando</i>	salto diretto salto indiretto
jje	<i>Etichetta</i>	salta se uguale / zero
jne	<i>Etichetta</i>	salta se non uguale / salta
js	<i>Etichetta</i>	zero se negativo
jns	<i>Etichetta</i>	salta se non negativo
jg	<i>Etichetta</i>	salta se maggiore
jge	<i>Etichetta</i>	salta se maggiore o uguale
jle	<i>Etichetta</i>	salta se minore o uguale
jl	<i>Etichetta</i>	salta se minore
...	...	...



Branch

Esempio di basso livello: ciclo for

Vediamo quanto è semplice tradurre il ciclo for in assembler:

```
for ( int i = 0; i < 10; i++ )  
    array[i] = 0;
```

L'indirizzo a cui fare riferimento viene incrementato (equivalente ad aumentare il contatore `i`)

Accesso diretto alla memoria all'elemento `i`-esimo dell'array

`rax` contiene l'indirizzo dell'array

`.L2:`

`mov`
`add`
`cmp`
`jne`

```
DWORD PTR [rax], 0  
rax, 4  
rax, rbp  
.L2
```

L'operatore di confronto; equivale a `rax-rbp` e imposta il flag `ZF` (nota: `rbp` contiene il valore di terminazione)

salta se non è uguale , ovvero salta a `.L2` se il flag `ZF` non è impostato



Branch

Esempio di basso livello: ciclo for

Questo è il flusso lineare di esecuzione nel corpo del ciclo

.L2:

↓
mov
add
cmp
jne

DWORD PTR [rax],
rax, 4
rax, rbp
.L2

Finché $rax \leq rbp$ (cioè $i < 10$) si verifica un *salto all'indietro* e il corpo viene eseguito di nuovo.

Quando il salto *non viene* eseguito e il flusso di controllo continua in seguito.





Branch

Esempio di basso livello: istruzione if

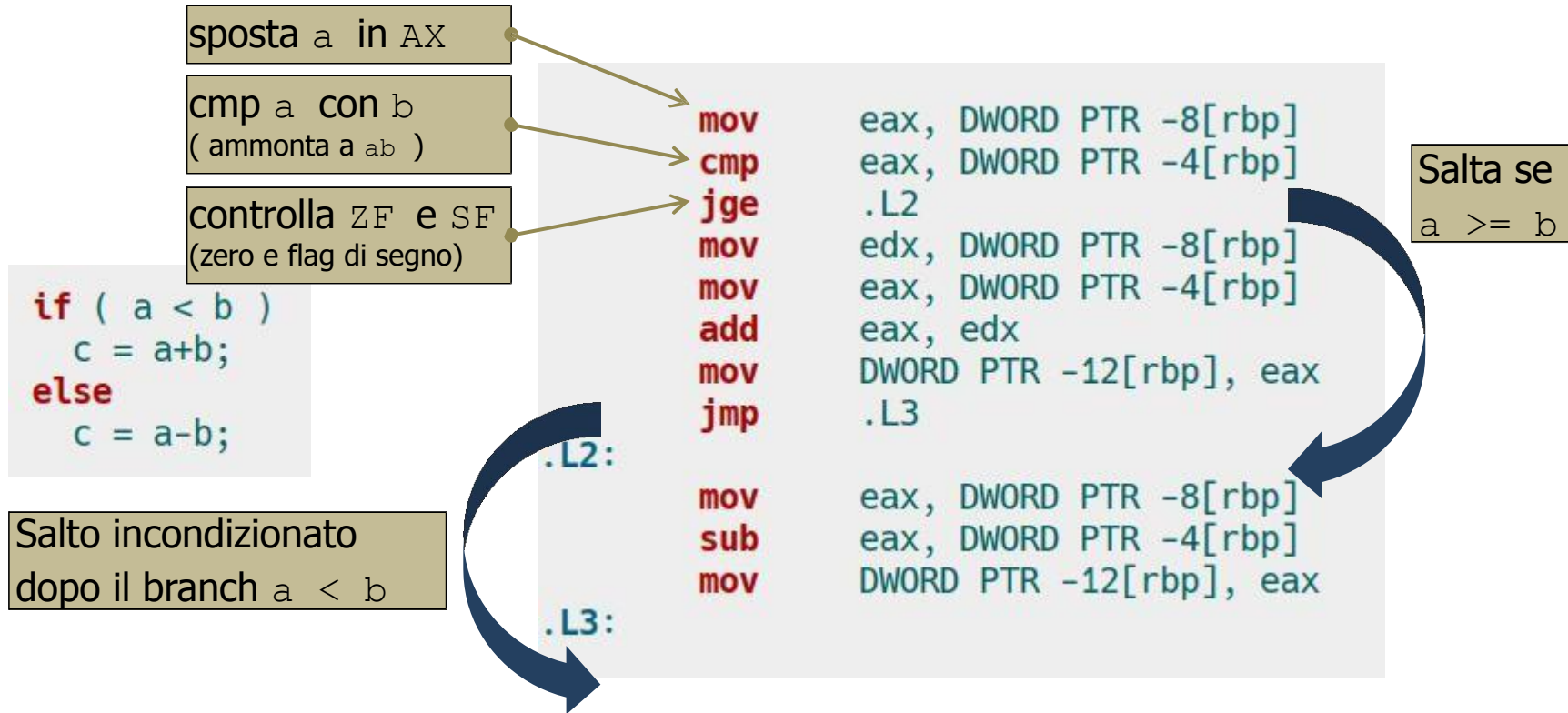
```
if ( a < b )  
    c = a+b;  
else  
    c = a-b;
```

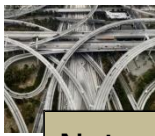
```
    mov     eax, DWORD PTR -8[rbp]  
    cmp     eax, DWORD PTR -4[rbp]  
    jge     .L2  
    mov     edx, DWORD PTR -8[rbp]  
    mov     eax, DWORD PTR -4[rbp]  
    add     eax, edx  
    mov     DWORD PTR -12[rbp], eax  
    jmp     .L3  
.L2:  
    mov     eax, DWORD PTR -8[rbp]  
    sub     eax, DWORD PTR -4[rbp]  
    mov     DWORD PTR -12[rbp], eax  
.L3:
```



Branch

Esempio di basso livello: istruzione if





Branch

Esempio di basso livello: istruzione if

Nota:

Il branch vero è quello più vicino alla condizione di test, mentre il branch falso viene raggiunto tramite un salto.

→ quando si scrive il codice, se possibile, bisogna prestare attenzione a ciò che è più probabile che sia vero, per preservare la **località del codice**.

(è possibile suggerire al compilatore quale branch sarà più probabilmente vero)

```
c = a+b;  
else  
c = a-b;
```

```
mov    eax, DWORD PTR -8[rbp]  
cmp    eax, DWORD PTR -4[rbp]  
jge    .L2  
mov    edx, DWORD PTR -8[rbp]  
mov    eax, DWORD PTR -4[rbp]  
add    eax, edx  
mov    DWORD PTR -12[rbp], eax  
jmp    .L3  
.L2:  
mov    eax, DWORD PTR -8[rbp]  
sub    eax, DWORD PTR -4[rbp]  
mov    DWORD PTR -12[rbp], eax  
.L3:
```



Branch

Flusso di dati condizionale

Abbiamo visto alcuni dettagli sul trasferimento condizionato di come *controllare il flusso* attraverso il semplice meccanismo del salto.

Tuttavia, questo potrebbe risultare piuttosto inefficiente nelle CPU moderne (ne parleremo più in dettaglio quando parleremo delle pipeline).

Un meccanismo diverso consiste nel **modificare in modo condizionale il flusso di dati**, che è il secondo meccanismo per l'esecuzione condizionale che abbiamo menzionato.



Branch

Flusso di dati condizionale

Il trasferimento condizionale del flusso di dati produce prestazioni molto elevate, ma è possibile solo in un piccolo sottoinsieme di casi;

fondamentalmente si tratta di valori semplici.

Un esempio tipico è, ad esempio, il valore assoluto di un risultato, o qualcosa di simile in cui ci si aspetta un risultato a valore singolo.

```
if ( a < b )  
    c = a+b;  
else  
    c = a-b;
```

Qui c contiene il risultato di un'operazione aritmetica molto semplice tra a e b .



Branch

Flusso di dati condizionale

```
if ( a < b )  
    c = a+b;  
else  
    c = a-b;
```

Compilato con
`gcc -O3 -march=native`



```
nota:  
ebx = a  eax = b
```

```
mov     edx, ebx  
lea     ecx, [rbx+rax]  
sub     edx, eax  
cmp     eax, ebx  
cmovg   edx, ecx
```

Un codice **molto** più breve ed efficiente!



Branch

Flusso di dati condizionale

	eax	ebx	ecx	edx
eax = b, ebx = a	b	a	-	-
mov edx, ebx	b	a	-	a
lea ecx, [rbx+rax]	b	a	a+b	a
sub edx, eax	b	a	a+b	a-b
cmp eax, ebx	confronta a e b; imposta ZF e CF (flag zero e carry)			
cmovg edx, ecx	sposta il valore di ecx in edx se la condizione (maggiore di) è soddisfatta			

La strategia è la seguente:

- (i) reg ecx contiene a+b mentre reg edx contiene ab
- (ii) la mossa condizionale controlla il risultato di cmp a, b (cioè il valore di ab)
- (iii) il contenuto di edx viene convertito in ecx per ogni evenienza.

Non sono state fornite istruzioni per il salto!



Branch

Outline

- a) Definizione di branch condizionali
- b) Flusso di esecuzione dipendente dai dati
- c) Flusso di dati dipendente dai dati
- d) Impatto dei branch condizionali sull'efficienza del codice**
- e) 4 esempi su come pulire/ristrutturare un codice
 1. branch condizionali all'interno dei cicli
 2. flussi di dati imprevedibili
 3. ordinamento di due array
 4. riempimento di una matrice



Branch

I pericoli dei branch condizionali

Per prevedere quale sarà il flusso di esecuzione, le CPU moderne sono dotate di un **predittore di diramazione**, ovvero un'unità interna di logica altamente sofisticata che indovina se un'istruzione di salto avrà successo o meno.

I migliori predittori di diramazione hanno una precisione pari al 95%; tuttavia, la previsione errata di diramazione, o branch miss, determina un'enorme perdita di prestazioni.

Le cifre tipiche per la penalità sono 10-30 cicli!

Questo perché più è lunga la pipeline, più avanti nel tempo sarà necessario analizzare il flusso, più sarà difficile e maggiore sarà la penalità per una previsione errata.

Esempio

*Supponiamo di avere 140 istruzioni in esecuzione e che 1 ogni 7 sia un'istruzione di diramazione. Qual è la probabilità che le pipeline **non** vengano svuotate con un predittore di diramazione corretto al 95%? E con uno al 90%?*



Branch

I pericoli dei branch condizionali

Esempio

*Supponiamo di avere 140 istruzioni in esecuzione e che 1 ogni 7 sia un'istruzione di diramazione. Qual è la probabilità che le pipeline **non** vengano svuotate con un predittore di diramazione corretto al 95%? E con uno al 90%?*

Qui l'idea è: la pipeline **non viene svuotata** solo se **non ci sono mispredizioni** su **nessuna** istruzione di branch.

- Istruzioni totali: 140
- 1 ogni 7 è branch $\Rightarrow$ numero di branch $= 140/7=20$

Se l'accuratezza del predittore è p , allora la probabilità di predire correttamente **tutti** i 20 branch (e quindi **mai** svuotare la pipeline per mispredizione) è:

$$P(\text{nessun flush}) = p^{20}$$

Predittore corretto al 95%

$$0.95^{20} \approx 0.3585 \Rightarrow 35.85\%$$

Predittore corretto al 90%

$$0.90^{20} \approx 0.1216 \Rightarrow 12.16\%$$

(Assumendo predizioni indipendenti tra loro)



Branch

I pericoli dei branch condizionali

Per prevedere quale sarà il flusso di esecuzione le CPU moderne sono dotate di un **predittore di diramazione** che indovina se un'istruzione

I migliori predittori di diramazione hanno una previsione errata di diramazione che riduce le prestazioni.

Le cifre tipiche per la penalizzazione sono:

Questo perché più è lunga l'istruzione, più il flusso, più sarà difficile e

Esempio

Supponiamo di avere 140 istruzioni.

Qual è la probabilità che le predizioni siano corrette al 95%? E con uno al 90%?

Ecco perché la seconda strategia che abbiamo visto, la **modifica condizionale del flusso di**

dati, è preferibile quando possibile e il compilatore cercherà di utilizzarla il più possibile:

non genera istruzioni di salto e il flusso di esecuzione è lineare e perfettamente prevedibile.

Tuttavia, come abbiamo detto, si applica solo a un limitato sottoinsieme di casi.



Branch

I pericoli dei branch condizionali

Le diramazioni condizionali dovrebbero essere evitate il più possibile all'interno dei cicli:

- spostandoli fuori dai loop e scrivendo loop più specializzati
 - esecuzione di variabili/quantità impostate preventivamente al di fuori dei cicli
 - utilizzando puntatori alle funzioni invece di selezionare funzioni all'interno del ciclo
 - sostituzione di branch condizionali con operazioni diverse
-



Branch

Outline

- a) Definizione di branch condizionali
- b) Flusso di esecuzione dipendente dai dati
- c) Flusso di dati dipendente dai dati
- d) Impatto dei branch condizionali sull'efficienza del codice
- e) 4 esempi su come pulire/ristrutturare un codice**
 - 1. branch condizionali all'interno dei cicli
 - 2. flussi di dati imprevedibili
 - 3. ordinamento di due array
 - 4. riempimento di una matrice



Branch

Il paesaggio

Ci sono fondamentalmente due casi possibili

(A) le variabili testate nelle condizioni **non** cambiano
durante il ciclo

il caso più semplice; con poca ristrutturazione, o addirittura niente, sei a posto

(B) le variabili testate nelle condizioni **cambiano**
durante il ciclo

il caso più difficile; probabilmente è necessario un lavoro significativo

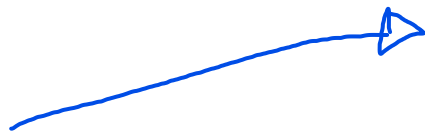
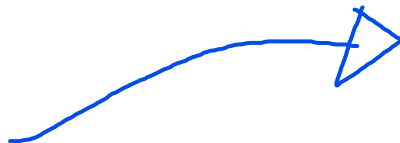


Branch

Pulisci i tuoi cicli dai branch

es 1: Prendere decisioni prima e fuori dal ciclo

```
for(i = 1; i < top; i++)  
{  
    if(case1 == 0) {  
        if(case2 == 0) {  
            if(case3 == 0)  
                result += i;  
            else  
                result -= i;  
        }  
        else {  
            if(case3 == 0)  
                result *= i;  
            else  
                result /= i;  
        }  
    }  
}
```



```
else {  
    if(case2 == 0) {  
        if(case3 == 0)  
            result += log10((double)i);  
        else  
            result -= log10((double)i);  
    }  
    else {  
        if(case3 == 0)  
            result *= sin((double)i);  
        else  
            result /= (sin((double)i) +  
                       cos((double)i));  
    }  
}
```



loop.c



Branch

Sollevamento (automatico) del loop

Quando i branch condizionali sono
all'interno del ciclo for e le
condizioni *non* dipendono
dall'iterazione del ciclo

Visto che value non viene mai cambiato
nel loop, è inutile fare mille controlli.
Meglio mettere l'if sopra e poi inserire i
for dentro l'if

Quindi il compilatore emetterà
versioni separate del ciclo for

```
int value = get_value_from_the_Universe();  
for(i = 1; i < top; i++) {  
    if ( value > 1 )  
        do_something();  
    else  
        do_something_else(); }
```

if (value > 1)

```
for(i = 1; i < top;  
i++) do_something();
```

else

```
for(i = 1; i < top;  
i++)  
do something else(); }
```



Branch

Pulisci i tuoi cicli dai branch

Se non ti fidi del tuo compilatore per eseguire il *sollevamento del ciclo* per te, puoi farlo manualmente o

- definire una funzione specializzata per ogni caso
- **prima** e **all'esterno** del ciclo imposta un puntatore di funzione alla funzione corretta

```
void (*func)(double *, int);  
<here make func pointing to the right place>  
double temp = 0;  
double result = 0;  
for(i = 1; i < top; i++)  
{  
    func( & temp, i);  
    result = temp;  
}
```



Branch

Pulisci i tuoi cicli dai branch

In questi casi è probabilmente molto meglio usare la costruzione `switch` invece di una foresta `if` , se è possibile tradurre i diversi test in un test sui valori di un singolo valore:

```
switch( case )  
{  
    case A: ...  
    break; case  
    B: ...  
        Break;  
    ...  
    default: ...  
}  
}
```



Infatti, il costrutto `switch` viene tradotto in una tabella statica di puntatori di codice a cui è possibile accedere direttamente:

```
table_of_addr[ #cases ] = { addr_A, addr_B, ... , addr_N, addr_default  
};  
if ( case > N )  
    jump to addr_default;  
jump to table_of_addr[ case ];
```



Branch

Rivedi il tuo codice

es. 2: ristrutturazione del codice per un flusso di dati imprevedibile

Considera il seguente frammento di codice

```
// generate random numbers
for (cc = 0; cc < SIZE; cc++)
    data[cc] = rand() % TOP;

// take action depending on their value
for (ii = 0; ii < SIZE; ii++)
{
    if (data[ii] >
        PIVOT) sum +=
        data[ii];
}
```



branch_prediction.c



Rivedi il tuo codice

Considera il seguente frammento di codice (*)

```
// generate random numbers
for (cc = 0; cc < SIZE; cc++)
    data[cc] = rand() % TOP;
qsort(data, SIZE, sizeof(int), compare);
// take action depending on their value
for (ii = 0; ii < SIZE; ii++)
{
    if (data[ii] >
        PIVOT) sum +=
        data[ii];
}
```

Almeno prima fai tutti gli If veri e poi tutti i falsi

(*) Naturalmente, si sta aggiungendo un overhead dovuto alla routine di ordinamento, quindi il tempo di esecuzione totale potrebbe essere ancora maggiore. Inoltre, si dovrebbero avere tutti i valori disponibili, quindi questo non funziona per gli streaming in tempo reale. Tuttavia, il punto qui è concentrarsi su come, in generale, sia meglio evitare le condizioni all'interno del ciclo, con qualsiasi possibile trucco o modifica nel flusso di lavoro.



Branch

Rivedi il tuo codice

Si può fare ancora meglio, senza aggiungere operazioni:

```
// generate random numbers
```

```
for (cc = 0; cc < SIZE; cc++)  
    data[cc] = rand() % TOP;
```

```
// take action depending on their value
```

```
for (ii = 0; ii < SIZE; ii++)  
{  
    t = (data[ii] - PIVOT - 1) >> 31;  
    sum += ~t & data[ii];  
}
```

Utilizza shift e cose per fare meglio



Branch

Rivedi il tuo codice

Un modo più semplice per trasformare questo codice

```
for (ii = 0; ii < SIZE; ii++)  
{  
    if (data[ii] > PIVOT)  
        sum += data[ii];  
}
```

è il seguente:

```
for (ii = 0; ii < SIZE; ii++)  
    sum += ( data[ii]>PIVOT ) * data[ii];
```



Rivedi il tuo codice

es 3: ristrutturazione del codice per l'ordinamento di due array

Hai 2 array, A e B, e vuoi scambiare i loro elementi in modo che

$$A[i] \geq B[i]$$

per tutti i .

Un'implementazione semplice sarebbe:

```
for (i = 0; i < SIZE; i++)  
{  
    if ( A[i] < B[i] )  
    {  
        t = B[i];  
        B[i] = A[i];  
        A[i] = t;  
    }  
}
```





Rivedi il tuo codice

Tuttavia, tale implementazione soffre esattamente dello stesso problema di cui abbiamo appena parlato.

Un modo alternativo per scrivere lo stesso codice, ma in uno stile più efficace, è:

```
for (i = 0; i < SIZE; i++)  
{  
    int min = A[i] > B[i] ? B[i] : A[i];  
    int max = A[i] >= B[i] ? A[i] : B[i];  
  
    A[i] = max;  
    B[i] = min;  
}
```



Branch

Rivedi il tuo codice

```
for (uint ii = 0; ii < SIZE; ii++)  
{  
    if ( B[ii] > A[ii] )  
    {  
        int t = A[ii];  
        A[ii] = B[ii];  
        B[ii] = t;  
    }  
}
```

standard

```
for (uint ii = 0; ii < SIZE; ii++)  
{  
    int register t = -(A[ii]<B[ii]);  
    int register x = A[ii]^B[ii];  
    A[ii] = A[ii]^(x & t);  
    B[ii] = B[ii]^(x & t);  
}
```

smart2

```
for (uint ii = 0; ii < SIZE; ii++)  
{  
    int max = (A[ii]>B[ii])? A[ii]:B[ii];  
    int min = (A[ii]>B[ii])? B[ii]:A[ii];  
    A[ii] = max;  
    B[ii] = min;  
}
```

smart

```
for (uint ii = 0; ii < SIZE; ii++)  
{  
    int d = A[ii]-B[ii];  
    d &= (d >> 31);  
    A[ii] = A[ii] - d;  
    B[ii] = B[ii] + d;  
}
```

smart3



Branch

Cambia il punto di vista

es. 4: sul fatto che il design e la semplicità siano la mossa migliore

A volte può essere utile anche solo cambiare punto di vista:

Questo codice inizializza una matrice $N \times M$ in modo che gli elementi int del triangolo in alto a destra siano impostati su 1.0 , le voci nella diagonale $i=j$ siano impostate su 0.0 e la parte in basso a sinistra sia impostata su -1.0

```
for ( j = 0; j < N; j++ )  
    for ( i = 0; i < M; i++ )  
    {  
        if ( i > j )  
            matrix[j][i] = 1.0;  
        else if ( i < j )  
            matrix[j][i] = -1.0;  
        else  
            matrix[j][i] = 0.0;  
    }
```



Branch

Cambia il punto di vista

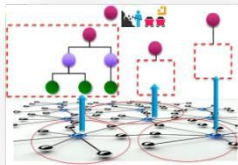
Può essere facilmente riscritto senza alcuna valutazione condizionale:

```
for ( int j = 0; j < N; j++ )
{
    int i;
    for ( i = 0; i < j; i++ )
        matrix[j][i] = -1.0;
    matrix[j][i] = 0.0;
    for ( i = j+1; i < M; i++ )
        matrix[j][i] = 1.0;
}
```

Una parola di cautela

Potrebbe essere davvero facile perdersi nell'"ottimizzazione", nel cercare ogni singola riga chiedendosi perché un trucco incredibile che - si è convinti - dovrebbe funzionare, in realtà non funziona.

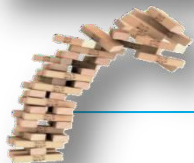
"Ottimizzazione" include anche l'ottimizzazione dei tuoi sforzi e del tuo tempo, quindi **ricorda sempre che gli ingredienti più importanti sono di gran lunga** :



Gli algoritmi che scegli



Il modello di dati che progetti



La qualità complessiva, la pulizia e la robustezza del tuo codice



Ottimizzazione 6 - I cicli

Principi della Programmazione Parallela
CdL Informatica - A.A. 2025/2026



Università
di Catania

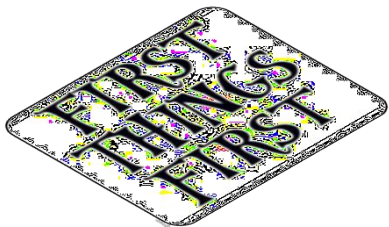


INAF
ISTITUTO NAZIONALE
DI ASTROFISICA

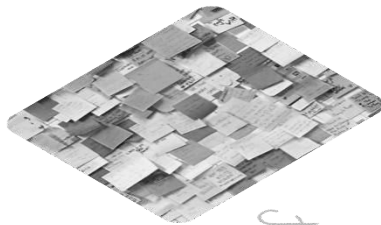


Ottimizzazione

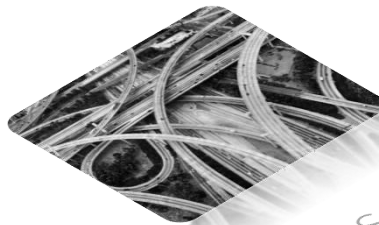
Outline



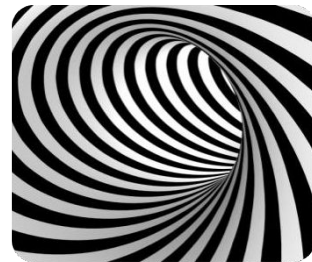
First
things
first



Cache &
Memory

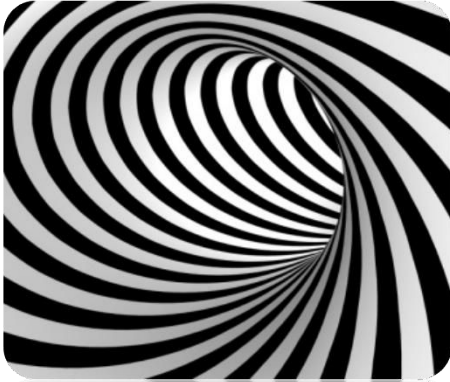


Branches



Loop

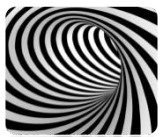
Outline



Tecniche
di loop



Precaricamento



Loop

Ottimizzazione dell'accesso alla cache nei cicli

Classificazione del ciclo

$$A_I = \frac{f(n)}{n}$$

Intensità aritmetica :
rapporto tra il numero
di operazioni eseguite
e la quantità di dati.

1. $O(N) / O(N)$

scansione di array, operazioni vettoriali 1D, ..

potenziale di ottimizzazione limitato

2. $O(N^2) / O(N^2)$

matrice $\times$ vettore, matr. trasp., matr. aggiungere, ...

qualche opportunità in più per
l'ottimizzazione.

3. $O(N^3) / O(N^2)$

matrice $\times$ matr,...

significativo potenziale di ottimizzazione



Loop

Accesso alla cache in cicli: $O(N)/O(N)$

Esempio

Cicli a 1 livello : prodotti scalari, addizioni vettoriali, moltiplicazione matrice-vettore sparso
Inevitabilmente legato alla memoria per *N molto grandi* ; in generale, i miglioramenti derivano dall'evitare operazioni *non necessarie e/o accessi ripetuti alla memoria* e dall'aumento **del riutilizzo dei dati**

[verificare la possibilità di fusione dei loop]

*è uno spreco $O(N) / O(N)$
fare due cicli diversi perché b va
due riaccessi B[i] in cache*

```
for(int i=0; i<N; i++) A[i] = B[i] × C[i]
```

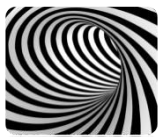


```
for(int i=0; i<N; i++)  
{  
    A[i] = B[i] × C[i]  
    Q[i] = B[i] + D[i]  
}
```

```
for(int i=0; i<N; i++) Q[i] = B[i] + D[i]
```

Fusione del loop: nella versione a destra, B viene richiamato dalla memoria solo una volta.





Loop

Accesso alla cache in cicli: $O(N^2)/O(N^2)$

Esempio

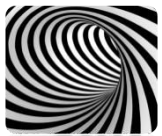
Cicli a 2 livelli : matrice
densa-vettore multiplo, matrice
trasposta, matrice addizionata, ...

I miglioramenti derivano ancora
una volta dall'aumento *del*
riutilizzo dei dati , dallo
sfruttamento *della località* e
dall'evitare operazioni e accessi
alla memoria non necessari.

```
for(int i=0; i < N; i++)  
    for(int j=0; j<N; j++)  
        C[i] += A[i][j] * B[j];
```

$N \times N$ $N \times N$ $N \times N$

$3 \times N^2$ accessi alla memoria



Loop

Accesso alla cache in cicli: $O(N^2)/O(N^2)$

Fase 1: Evitare caricamenti non necessari

```
for(int i=0; i < N; i++)  
    for(int j=0; j<N;  
j++)
```

```
    C[i] += A[i][j] * B[j];
```

*Perché ricaricare C[i] ?
a ogni iterazione*

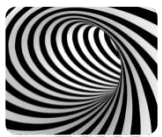


```
for(int i=0; i < N; i++)  
{
```

```
    c_temp = C[i];  
    for(int j=0; j < N;  
j++)
```

Ora è chiaro per il compilatore che $C[i]$ deve essere caricato e conservato solo 1 volta

$2 \times N^2 + N$ accessi alla memoria



Loop

Accesso alla cache in cicli: $O(N^2)/O(N^2)$

Fase 2:

Se i tuoi registri vettoriali possono contenere 4 numeri float, allora ha senso scegliere $m=4$ (o un suo multiplo). In questo modo, una singola istruzione della CPU calcola 4 righe della matrice in un colpo sol

Srotolare il ciclo esterno e fonderlo nel ciclo interno
(*unroll & jam*); esiste il potenziale per la vettorizzazione.

```
for(int i=0; i < N; i++)
    for(int j=0; j<N;
j++)
        C[i] += A[i][j] * B[j];
```

$$N^2 \times (1 + 1/m) + N$$

→

```
for(int i=0; i < N; i += m)
    for(j = 0; j < N; j++) {
        b_temp = B[j];
        C[i] += A[i][j] * b_temp;
        C[i+1] += A[i+1][j] * b_temp;
        ...
        C[i+m] += A[i+m][j] * b_temp; }
}
```

$N \times N/m$

$N \times N$

N



Loop

Nota: srotolamento e registrazione delle fuoriuscite

m troppo grande nell'esempio precedente, mentre la CPU di destinazione non ha registri sufficienti per conservare tutti gli operandi necessari, provoca un "gonfiamento del codice".

In questo caso, la CPU deve trasferire il contenuto dei registri nella cache e viceversa, rallentando il calcolo.

-> imparare a ispezionare il **log del compilatore**

Un corpo di loop troppo involuto e oscuro potrebbe impedire al compilatore di eseguire in modo efficace le ottimizzazioni *unroll & jam* mirate alla CPU su cui viene eseguito.

-> **sforzo di codifica manuale per chiarire il codice**

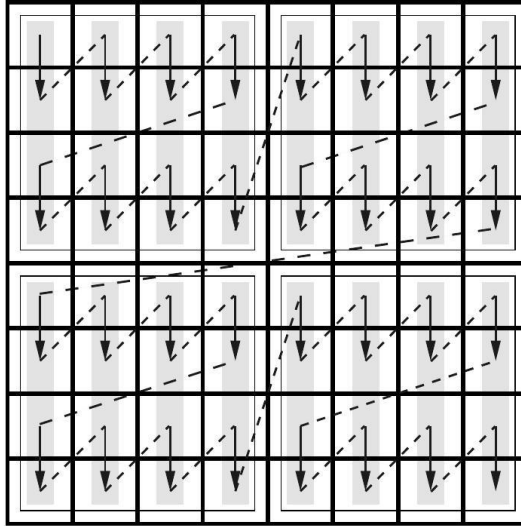
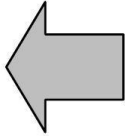
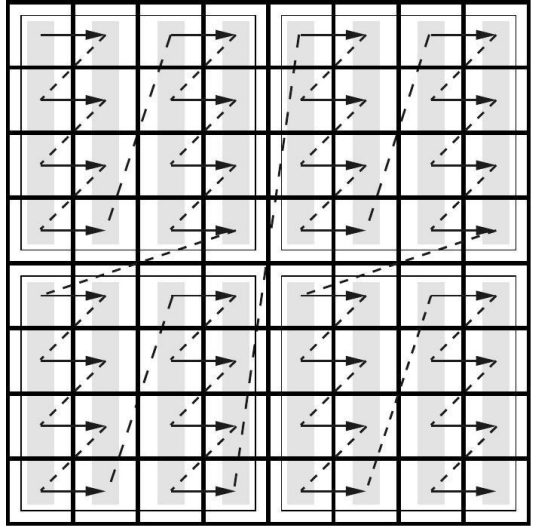
-> **suggerimenti/direttive al compilatore**

(le direttive generalmente non sono portabili tra diversi compilatori)



Loop

Accesso alla cache in cicli: $O(N^2)/O(N^2)$



Fase 3:
Sfrutta appieno
la località dei dati
referenziati;
accedendo agli
array 2D per
blocchi



Loop

Srotolamento del loop

Lo svolgimento del ciclo (loop unrolling) è una trasformazione fondamentale del codice che solitamente contribuisce in modo significativo a migliorare le prestazioni del codice:

- Riduce il sovraccarico del ciclo (aggiornamento del contatore, ramificazione)
 - Espone *il percorso dei dati critici* e le dipendenze
 - Aiuta a sfruttare l'ILP, soprattutto in caso di aliasing della memoria
-



Loop

Srotolamento del loop

Esaminiamo questa semplice *riduzione* :

```
for ( int i=0; i<N; i++ )
```

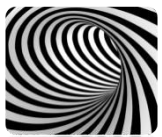
```
S += A[i];  
-O3 -march=native
```

-O0

```
.L3:  
# reduction.c:41:    acc = acc OP array[ii];  
    mov     rax, QWORD PTR -24[rbp] # ii  
    lea     rdx, 0[0+rax*8]  
    mov     rax, QWORD PTR -48[rbp] # array  
    add     rax, rdx  
    movsd   xmm0, QWORD PTR [rax]  
    movsd   QWORD PTR -56[rbp], xmm0  
    fld     QWORD PTR -56[rbp]  
# reduction.c:41:    acc = acc OP array[ii];  
    fld     TBYTE PTR -16[rbp]      # acc  
    faddp   st(1), st  
    fstp    TBYTE PTR -16[rbp]      # acc  
# reduction.c:40:    for ( uLint ii = 1; ii < N; ii++ )  
    add     QWORD PTR -24[rbp], 1   # ii,  
.L2:  
# reduction.c:40:    for ( uLint ii = 1; ii < N; ii++ )  
    mov     rax, QWORD PTR -24[rbp] # ii  
    cmp     rax, QWORD PTR -40[rbp] # N  
    jb     .L3
```

```
.L3:  
# reduction.c:41:    acc = acc OP array[ii];  
    fadd     QWORD PTR [rax] # array  
    add     rax, 8           #  
# reduction.c:40:    for ( uLint ii = 1; ii < N; ii++ )  
    cmp     rdx, rax         # N  
    jne     .L3  
    ret
```

Con l'ottimizzazione attivata, il compilatore gestisce molto meglio il sovraccarico del ciclo, ma non ottimizza in alcun modo le operazioni FP.



Loop

Srotolamento del loop

Se compiliamo *esattamente lo stesso codice* ma utilizzando `int` come tipo di dati invece di `double` , otteniamo un risultato completamente diverso:

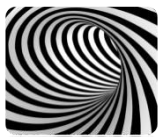
-00

-03 -march=native

```
.L3:
# reduction.c:43:    acc = acc OP array[ii];
    movq    -8(%rbp), %rax # ii
    leaq    0(,%rax,4), %rdx
    movq    -32(%rbp), %rax # array
    addq    %rdx, %rax
    movl    (%rax), %eax
    movl    %eax, %eax
# reduction.c:43:    acc = acc OP array[ii];
    addq    %rax, -16(%rbp)
# reduction.c:42:    for ( uInt ii = 1; ii < N; ii++ )
    addq    $1, -8(%rbp)    #, ii
.L2:
# reduction.c:42:    for ( uInt ii = 1; ii < N; ii++ )
    movq    -8(%rbp), %rax # ii
    cmpq    -24(%rbp), %rax # N
    jb      .L3
```

```
.L4:
# reduction.c:43:    acc = acc OP array[ii];
    vmovdqu 4(%rax), %ymm0
    addq    $32, %rax
    vpmovzxdq    %xmm0, %ymm1
    vextracti128 $0x1, %ymm0, %xmm0
    vpmovzxdq    %xmm0, %ymm0
# reduction.c:43:    acc = acc OP array[ii];
    vpaddq    %ymm0, %ymm1, %ymm0
    vpaddq    %ymm0, %ymm2, %ymm2
    cmpq    %rdx, %rax
    jne      .L4
```

Ora il compilatore opta per la completa *vettorializzazione* del ciclo, con un impatto molto forte sulle prestazioni!!



Loop

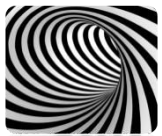
Srotolamento del loop

Perché il compilatore sceglie di ottimizzare il ciclo quando i dati sono di tipo `int` e non lo fa quando i dati sono di tipo `double` ?

Ciò è dovuto al fatto che, come abbiamo [visto](#) il compilatore NON è libero di ristrutturare il codice che gestisce i numeri in virgola mobile.

Poiché la matematica con virgola mobile non è associativa, la modifica dell'ordine esatto delle operazioni nel codice potrebbe (da quanto il compilatore può giudicare in fase di compilazione) modificare la correttezza dei calcoli in fase di esecuzione.

Come abbiamo visto, cambiare l'ordine delle operazioni ha ovviamente un impatto sul risultato. Dal punto di vista del compilatore, potresti aver scelto un determinato flusso di lavoro proprio perché *sai* che è il più corretto in relazione ai dati a [cui verrà applicato!](#)



Loop

Srotolamento del loop

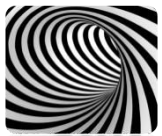
A questo punto, ci rimane la responsabilità di ottimizzare questo semplice codice. Dato che lo stiamo analizzando continuamente nell'ordine naturale della memoria, la cache non rappresenta un problema.

Il nostro obiettivo è ristrutturare il codice in modo che il compilatore possa sfruttare l'ILP (*Instruction-Level Parallelism*) della CPU .

```
for ( int i=0; i<N; i++ )  
    S += A[i];
```



pipeline/reduction



Loop

Fase 1: srotolare 2×1

Il nostro primo tentativo è quello di ridurre il sovraccarico del ciclo e di esporre un certo parallelismo tra i dati elaborando esplicitamente 2 elementi per iterazione.

```
for ( int i=0; i<N; i++ )  
    S = S OP A[i];
```



```
for ( int i=0; i<N-2; i+=2 )  
    S = (S OP A[i]) OP A[i+1]  
    ;
```



Loop

Fase 1: srotolare 2×1

Il nostro primo tentativo è quello di ridurre il sovraccarico del ciclo e di esporre un certo parallelismo tra i dati elaborando esplicitamente 2 elementi per iterazione.

```
for ( int i=0; i<N; i++ )  
    S = S OP A[i];
```



```
for ( int i=0; i < N-2; i+=2 )  
    S = (S OP A[i]) OP A[i+1] ;
```

Nota: quando si esegue lo srotolamento, bisogna sempre tenere conto delle iterazioni finali che rimarrebbero indietro. Un modo comune per farlo per il fattore di srotolamento U (solitamente U varia in $[2..16]$ di un ciclo con N iterazioni) è:

```
int N_ = (N/U)*U;      // per costruzione questo è il più grande multiplo  
                        // di U  
for (int i = 0; i < N_ ; i += U)  
)  
    operazioni_di_iterazione ;  
for (int i = N_ ; i < N ; i += U)  
)
```



Loop

Fase 2: srotolamento 2×1 + rimescolamento

Esploriamo cosa succede se apportiamo una modifica semantica al nostro codice, sfruttando semplicemente il fatto che il nostro OP è associativo.

```
for ( int i=0; i<N-2; i+=2 )  
    S = ( S OP A[i] ) OP A[i+1] ;
```



```
for ( int i=0; i<N-2; i+=2 )  
    S = S OP ( A[i] OP A[i+1] )  
    ;
```



Loop

Fase 2: srotolamento 2×1 + rimescolamento

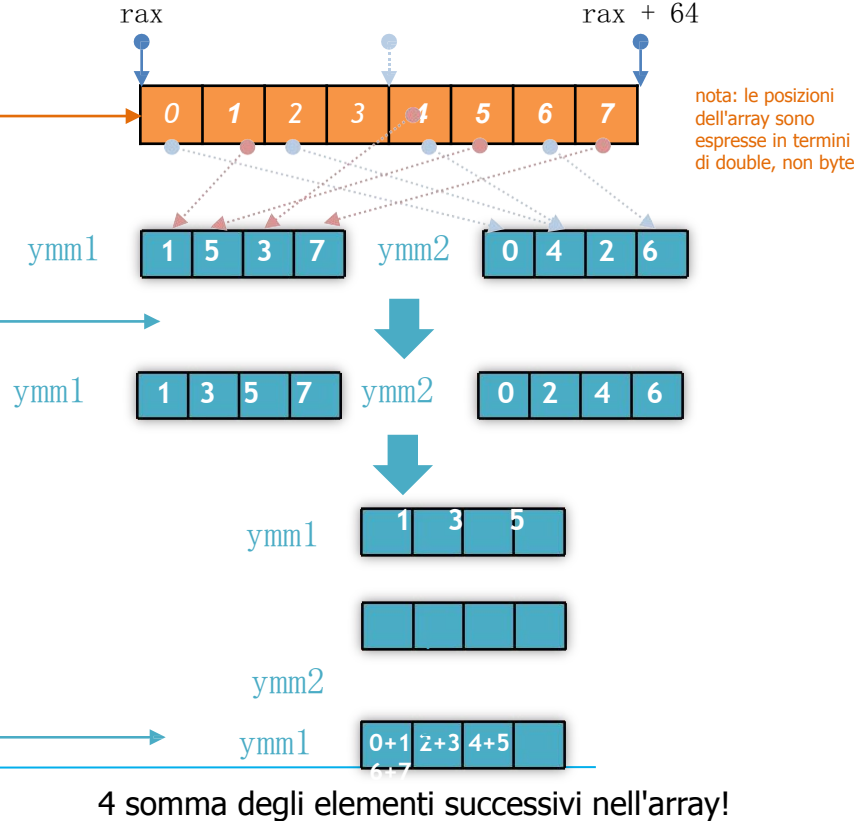
```
for ( int i=0; i<N-2; i+=2 )  
    S = S OP ( A[i] OP A[i+1]  
    );
```

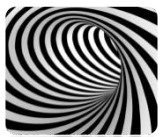


.L33:

```
vmovupd ymm4, YMMWORD PTR [rax]  
add     rax, 64  
vunpcklpd ymm1, ymm4, YMMWORD PTR -32[rax]  
vunpckhpd ymm2, ymm4, YMMWORD PTR -32[rax]  
vpermpd ymm1, ymm1, 216  
vpermpd ymm2, ymm2, 216  
vaddpd ymm1, ymm1, ymm2  
vaddsd xmm0, xmm0, xmm1  
vunpckhpd xmm3, xmm1, xmm1  
vextractf128 xmm1, ymm1, 0x1  
vaddsd xmm0, xmm0, xmm3  
vaddsd xmm0, xmm0, xmm1  
vunpckhpd xmm1, xmm1, xmm1  
vaddsd xmm0, xmm0, xmm1
```

Vengono elaborati 8 double per iterazione; grazie alla riassociazione delle operazioni, il compilatore può riorganizzare le operazioni in modo più efficiente





Loop

Fase 2: srotolamento 2×1 + rimescolamento

Per essere più chiari: solo perché abbiamo riassociato l'espressione matematica nel ciclo,

da $S = (S \text{ OP } A[i]) \text{ OP } A[i+1]$;

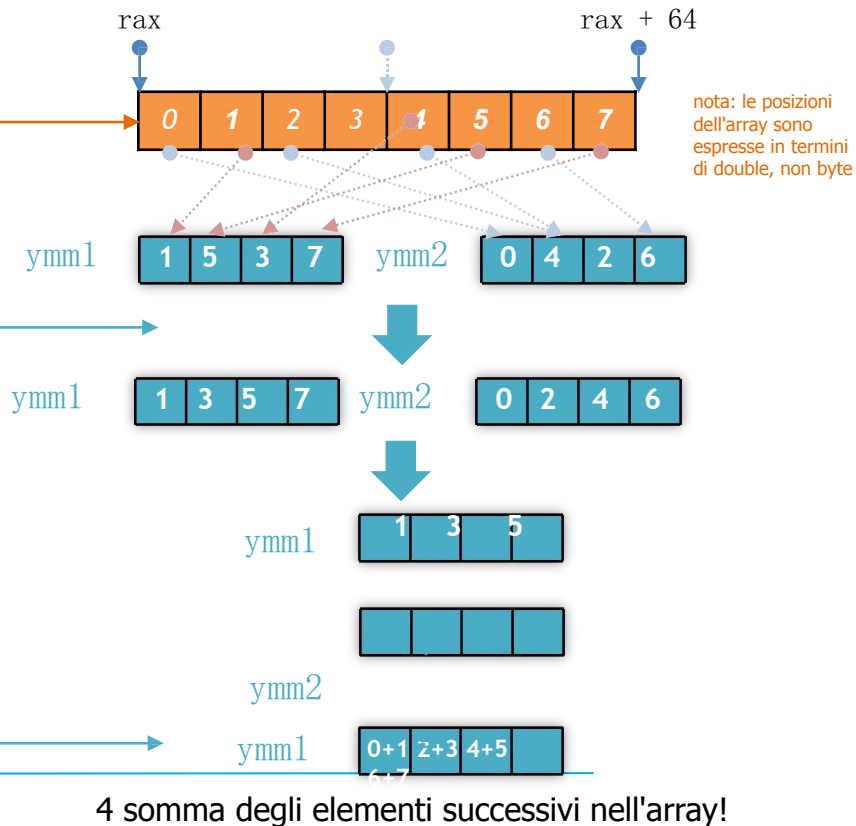
a $S = S \text{ OP } (A[i] \text{ OP } A[i+1])$;

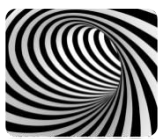
il compilatore ha il diritto di sfruttare il fatto che *nella semantica che ora stiamo dando*, l'ordine delle operazioni sarà

Somma = (((([0]+[1])) + (([2]+[3])) + ([4]+[5])) + ([6]+[7]))...

Utilizza shift e cose per fare meglio

E il risultato è quello che abbiamo appena discusso, più efficiente di quello che abbiamo ottenuto con il codice precedente





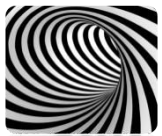
Loop

Accesso alla cache in cicli: $O(N^3)/O(N^2)$

Questi algoritmi (ad esempio: moltiplicazione matrice-matrice o diagonalizzazione di matrice densa) sono ottimi candidati per ottimizzazioni che portano le prestazioni flop/s molto vicine al picco teorico (infatti, MMM è al centro di `linpack`).

Tailing, unroll&jam + vettorizzazione delle operazioni, riorganizzazione delle operazioni per sfruttare le pipeline della CPU e capacità out-of-order, sono tutti utilizzati da **librerie estremamente specializzate** .

È un'idea brillante quella di collegare queste librerie invece di sviluppare un proprio algoritmo, a meno che non si debbano soddisfare esigenze molto particolari.



Loop

Esempio $O(N^3)/O(N^2)$

La moltiplicazione matrice-matrice è un'operazione molto comune nell'HPC.

Sebbene esistano librerie altamente ottimizzate che svolgono questo compito, si tratta di un caso di studio molto classico e utile per comprendere il tiling dei loop e gli algoritmi cache-oblivious.

Partiamo dalla definizione del problema.

Date 2 matrici, A e B, aventi rispettivamente (m, n) e (n, p) righe e colonne, il loro prodotto è definito come la matrice $C(m, p)$

$$C_{i,j} = \sum_{k=0}^n A_{i,k} \times B_{k,j}$$



Loop

Esempio $O(N^3)/O(N^2)$

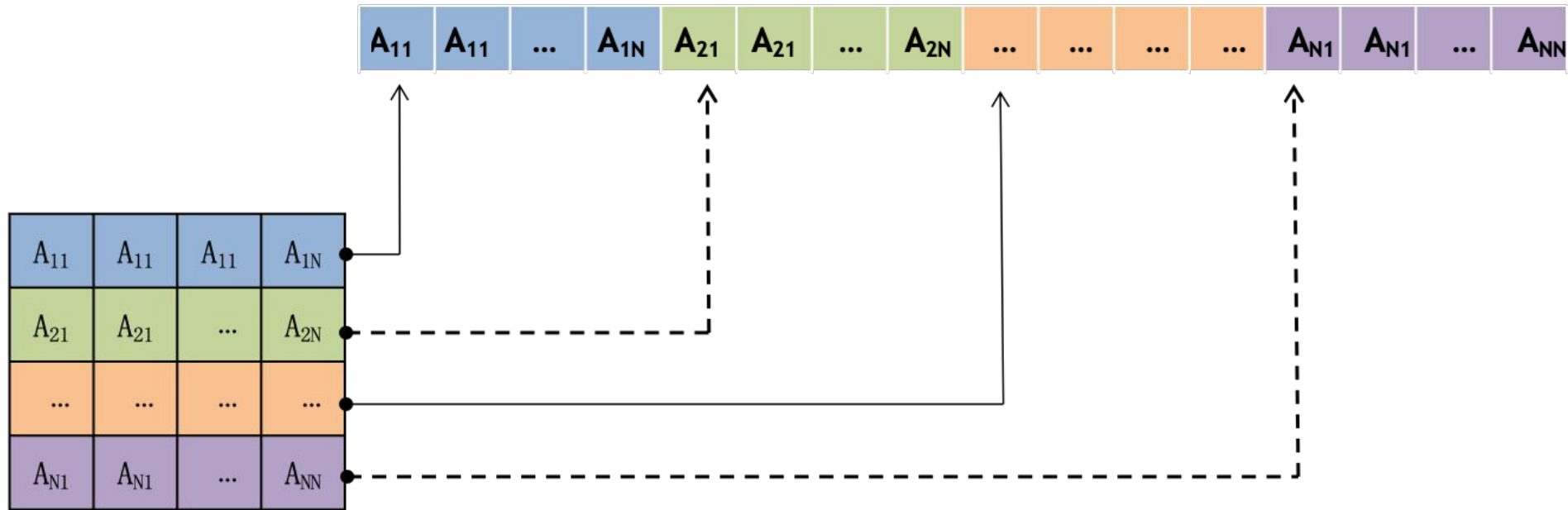
$$C_{i,j} = \sum_{k=0}^n A_{i,k} \times B_{k,j}$$

Una possibile implementazione semplice e ovvia di questo algoritmo è la seguente:

```
for ( int i = 0; i < m; i++ ) // attraversa le righe A (e C)
    for (int k = 0; k < p; k++ )           // attraversa le righe di B ( Ac =
                                                Br )
        for 0; j < n; j++ )
            (int j =
                A[i][j] * B[j][k];
C[i][k] +=
```

Nota: come una matrice viene memorizzata nella memoria

Ricorda il fatto ovvio che la tua memoria è un flusso continuo di byte unidimensionali.

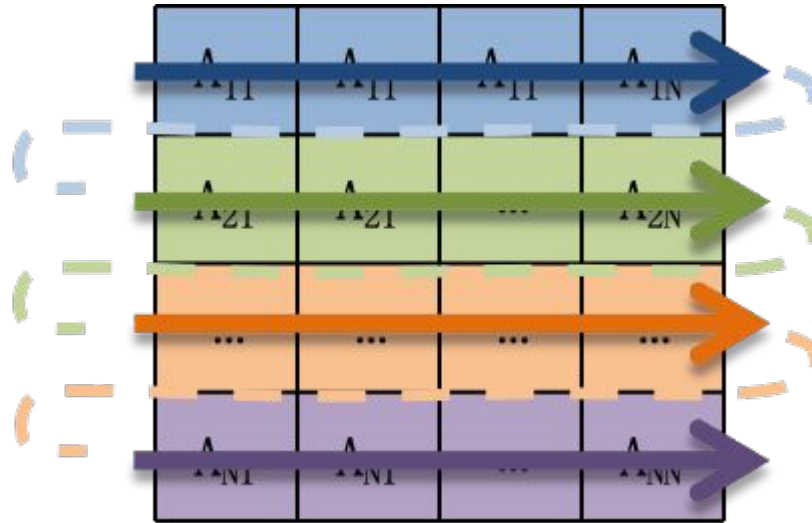


Questa convenzione è la convenzione C/C++, etichettata come *ordine row-major*. Si noti che la convenzione Fortran è opposta, con le colonne contigue in memoria (*column-major*).

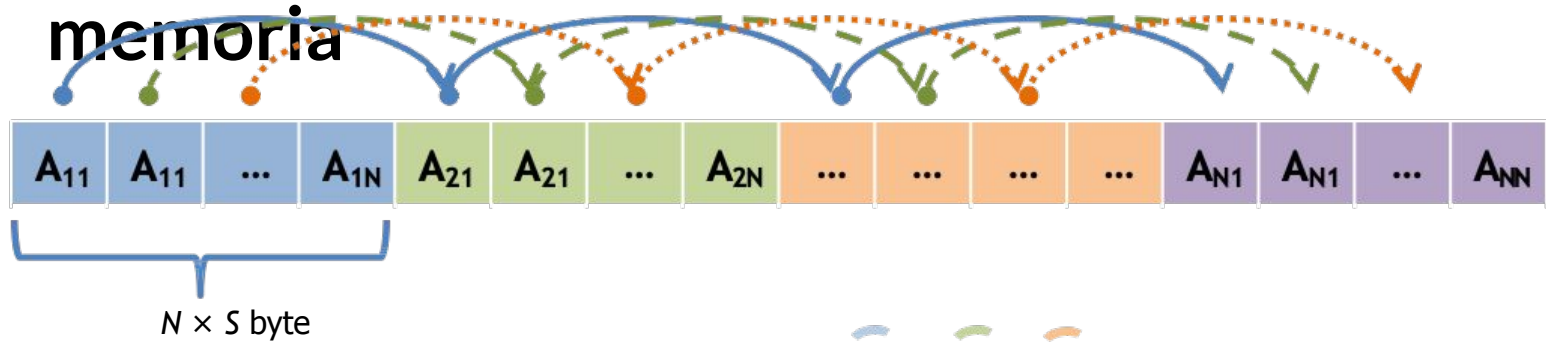
Nota: come una matrice viene memorizzata nella memoria



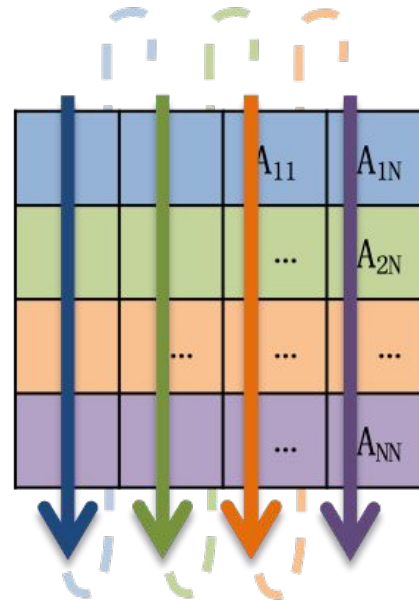
Quindi,
attraversare la
matrice nello
stesso ordine di
riga equivale ad
attraversare la
memoria in ordine
contiguo.



Nota: come una matrice viene memorizzata nella memoria



Mentre attraversare la matrice nell'ordine opposto delle colonne equivale a saltare nella memoria di N posizioni, ovvero $N \times S$ byte dove S è la dimensione di ciascun elemento.





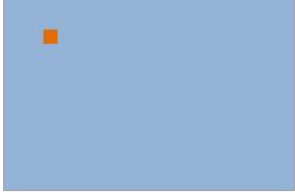
Loop

Esempio $O(N^3)/O(N^2)$

Rappresentazione della matrice

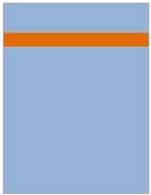
Rappresentazione della memoria

C



=

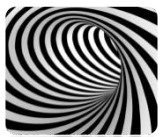
A



×

B





Loop

Esempio $O(N^3)/O(N^2)$

L'implementazione ingenua presenta un evidente problema di località dei dati per matrici sufficientemente grandi.

Per ogni elemento di C , è possibile che tutti gli accessi a B risultino in un cache miss. Quindi, potremmo avere cache miss mnp/L (se L è la capacità di riga della cache in termini di tipo di dati utilizzato) solo per attraversare B .

Il numero totale di *miss* previsti è:

$$\begin{aligned} & mp/L + \\ & mnp/L \\ & + mnp \end{aligned}$$

C viene attraversato una volta

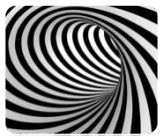
A viene scansionato interamente p volte

B è accessibile scarsamente p volte

In effetti, l'implementazione ingenua non viene mai utilizzata per grandi moltiplicazioni di matrici: poiché sono richiesti $2mnp$ flop, si ha quasi un cache miss per ogni flop.

Come possiamo risolvere questo problema?

Trasporre la matrice B prima di entrare nel ciclo dovrebbe alleviare il problema; sebbene la trasposizione richieda un po' di lavoro aggiuntivo, per matrici sufficientemente grandi si ottiene comunque un miglioramento delle prestazioni.



Loop

Esempio $O(N^3)/O(N^2)$

Una strategia diversa potrebbe consistere nello **scambiare i due cicli interni:**

```
for (int i = 0; i < m; i++)  
    for (int k = 0; k < p; k++)  
        for (int j = 0; j < n; j++)  
            C[i][k] += A[i][j] *  
                B[j][k];
```

```
for (int i = 0; i < m; i++)  
    for (int j = 0; j < n; j++)  
        for (int k = 0; k < p; k++)  
            C[i][k] += A[i][j] *  
                B[j][k];
```

```
// attraversa le righe A (e C)  
// attraversa le righe di B (Ac =  
Br )
```

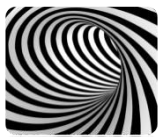
Ora abbiamo ancora molti miss di cache dovuti al fatto che stiamo ricaricando $C[i][k]$ molte volte (volte Ac).

Ora ci aspettiamo che $mnp/L + \text{loop}$
su C $mnp/L + \text{loop}$ su A mnp/L
loop su B

cache miss. Quindi, rispetto al precedente schema di annidamento ci aspettiamo di avere

$\sim mnp$
meno miss di cache.





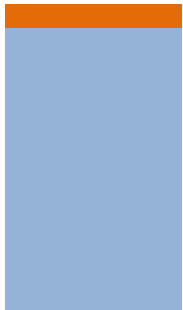
Loop

Esempio $O(N^3)/O(N^2)$

Possiamo fare ancora meglio ottimizzando sia gli accessi alla memoria sia la contiguità dei dati tramite *il tailing* dei loop:



=



×



Per calcolare una singola riga in C dobbiamo accedere alla riga corrispondente in A p volte. Nell'ipotesi che A, B e C non possano stare in memoria, ogni volta la cache verrà svuotata e la riga di A non sarà più presente.

Ciò equivale ad avere n/L mancanze obbligatorie per ogni colonna di B, ovvero np/L mancanze nella cache per ogni riga di C, come abbiamo già calcolato.

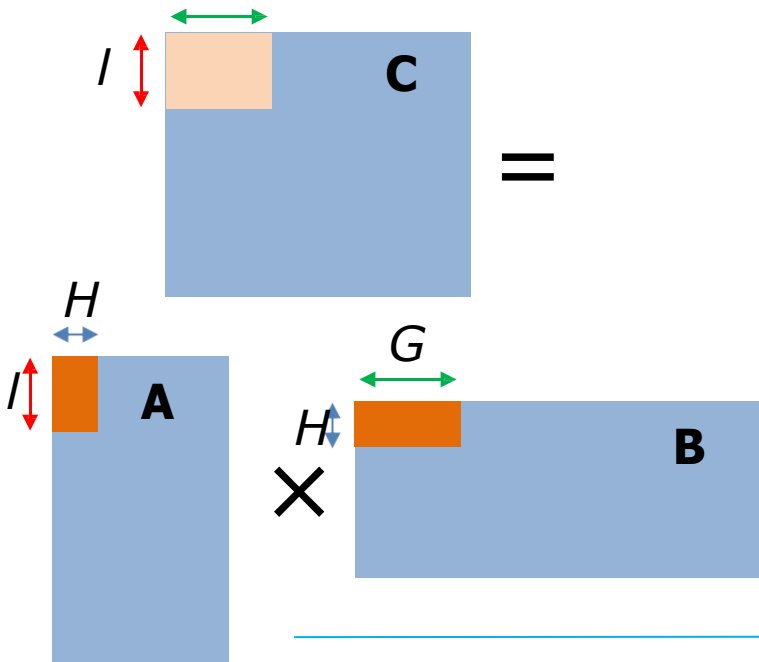
Lo stesso vale per le colonne B' e così via...



Loop

Esempio $O(N^3)/O(N^2)$

Possiamo fare ancora meglio ottimizzando sia gli accessi alla memoria sia la contiguità dei dati tramite *il tailing* dei loop:



Se invece mantenessimo nella cache un segmento della linea di A, riutilizzandolo contro le colonne di B – o, meglio, contro una sezione di colonna alta quanto è grande il segmento di linea (freccie blu nella figura) – potremmo ridurre notevolmente la quantità di cache miss per elemento di C.

Attraversando A e B per *blocchi* come in figura si possono accumulare risultati parziali nell'area C corrispondente, diminuendo di un fattore il numero di cache miss

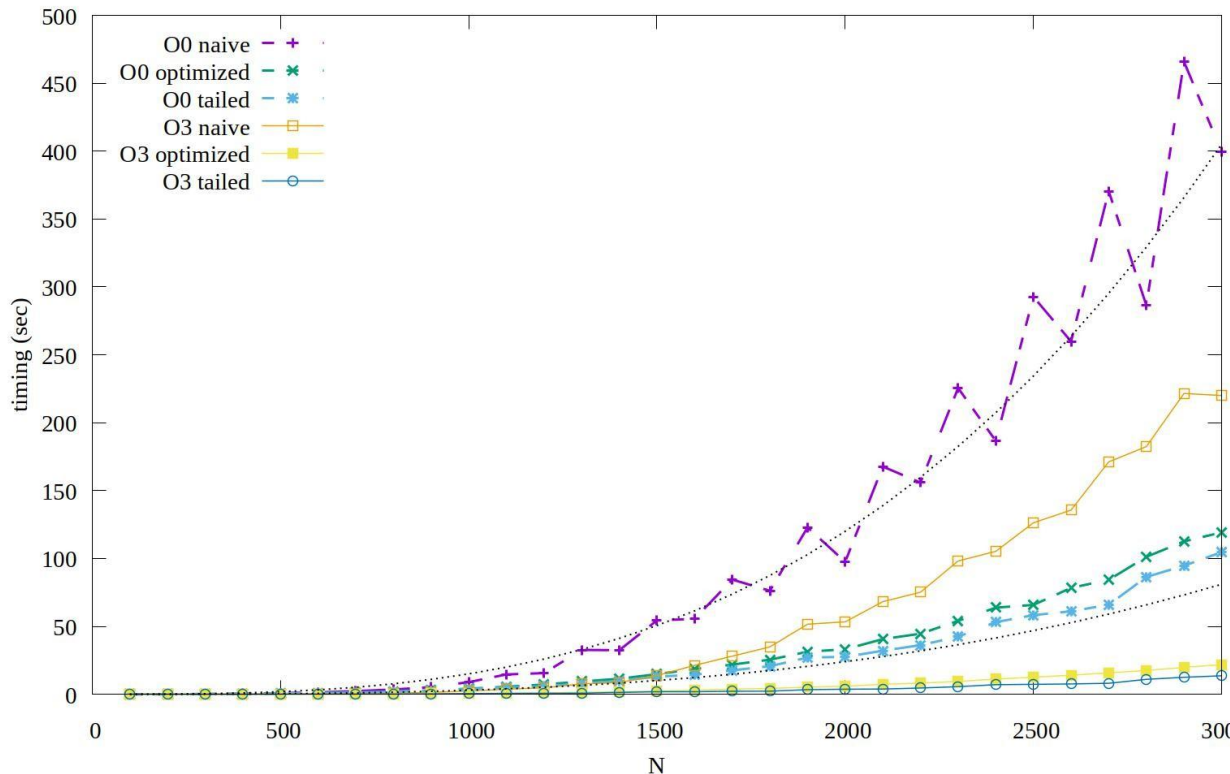
$$L / (I \times a \times g)$$

dove L è la capacità della cache e sono i fattori di blocco. Con valori standard, questo valore diventa dell'ordine di 0,001.



Loop

Risultati della moltiplicazione di matrici



Tempo di esecuzione di diverse implementazioni con e senza ottimizzazione del compilatore

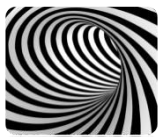
i risultati sono per il caso di matrice quadrata 2 di dimensione N

Naïve: l'implementazione del libro scolastico

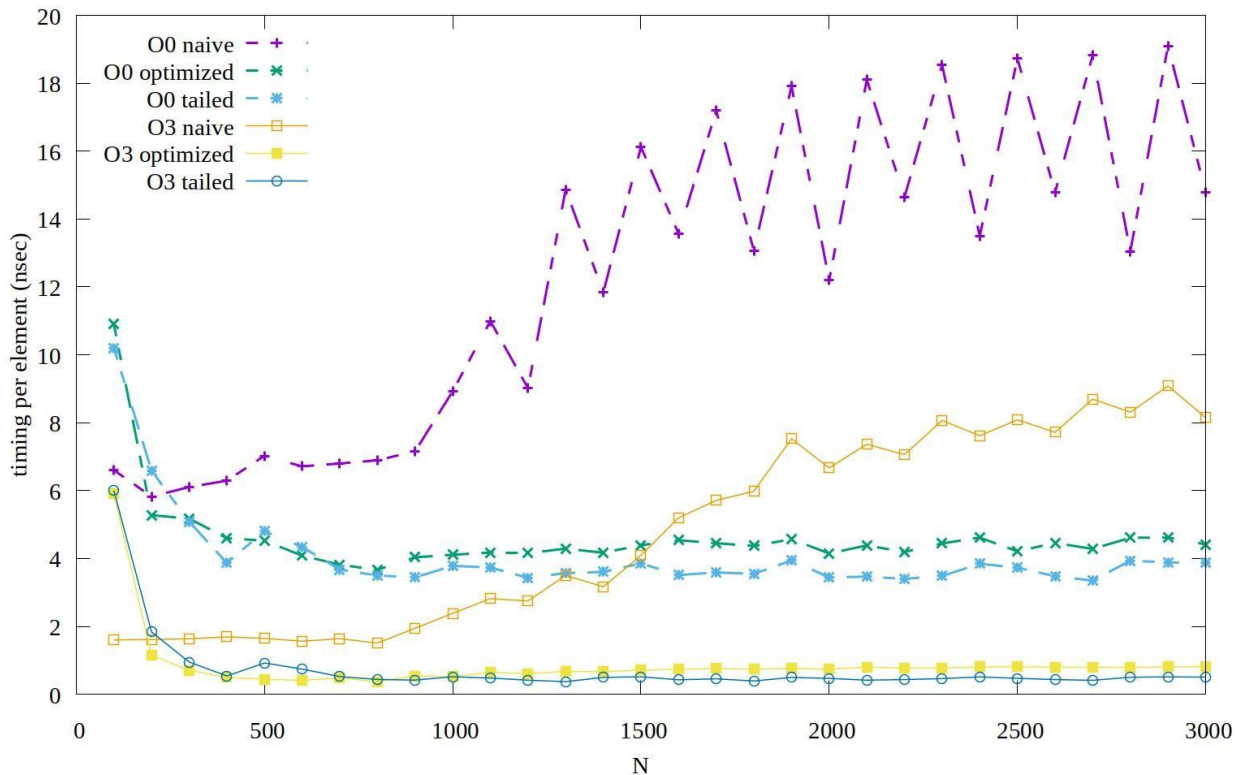
Ottimizzato: cicli interni [scambiati](#)

Tailed: MM per blocchi

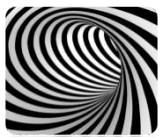
In questo grafico e nei seguenti le linee tratteggiate indicano `-O0` e la linea continua indica `-O3 -march=native` (è stato utilizzato solo `gcc`)



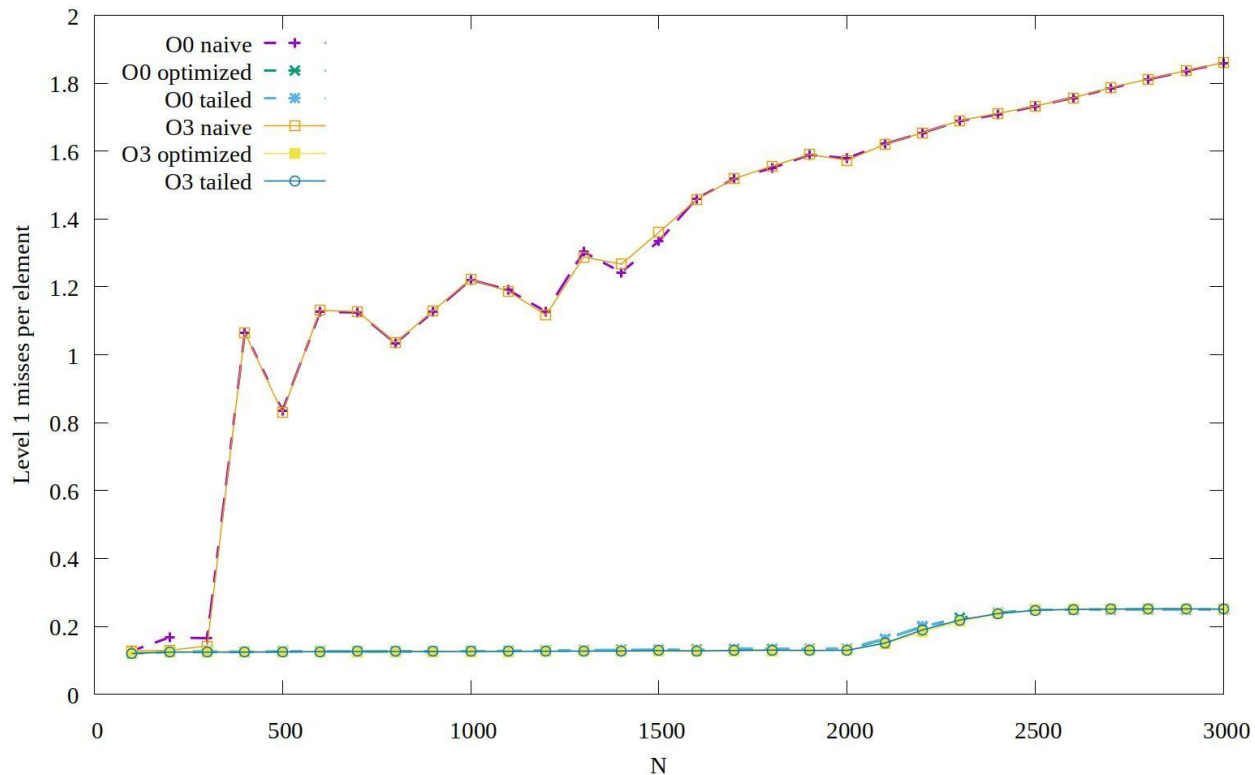
Risultati della moltiplicazione di matrici



**Tempo per
elemento a cui
si accede (N^3)**



Risultati della moltiplicazione di matrici

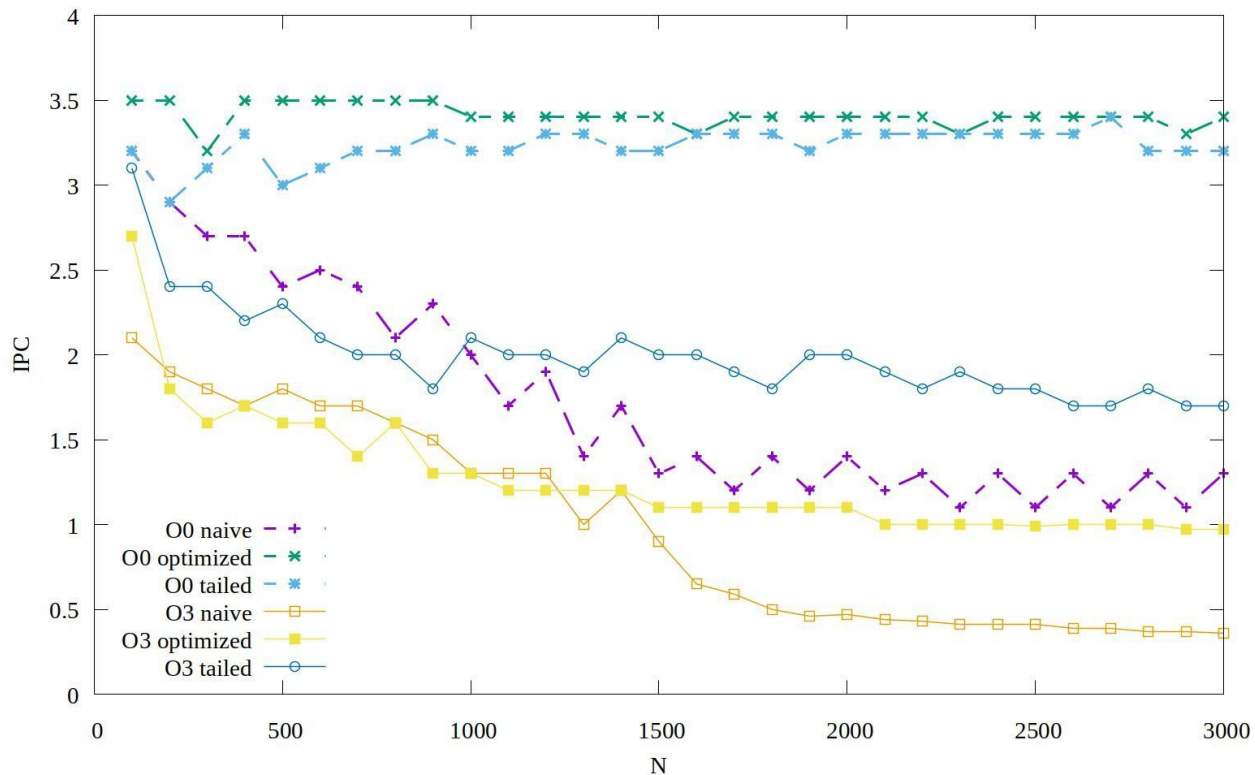


**Livello 1 Miss
nella cache dei
dati per
elemento**



Loop

Risultati della moltiplicazione di matrici



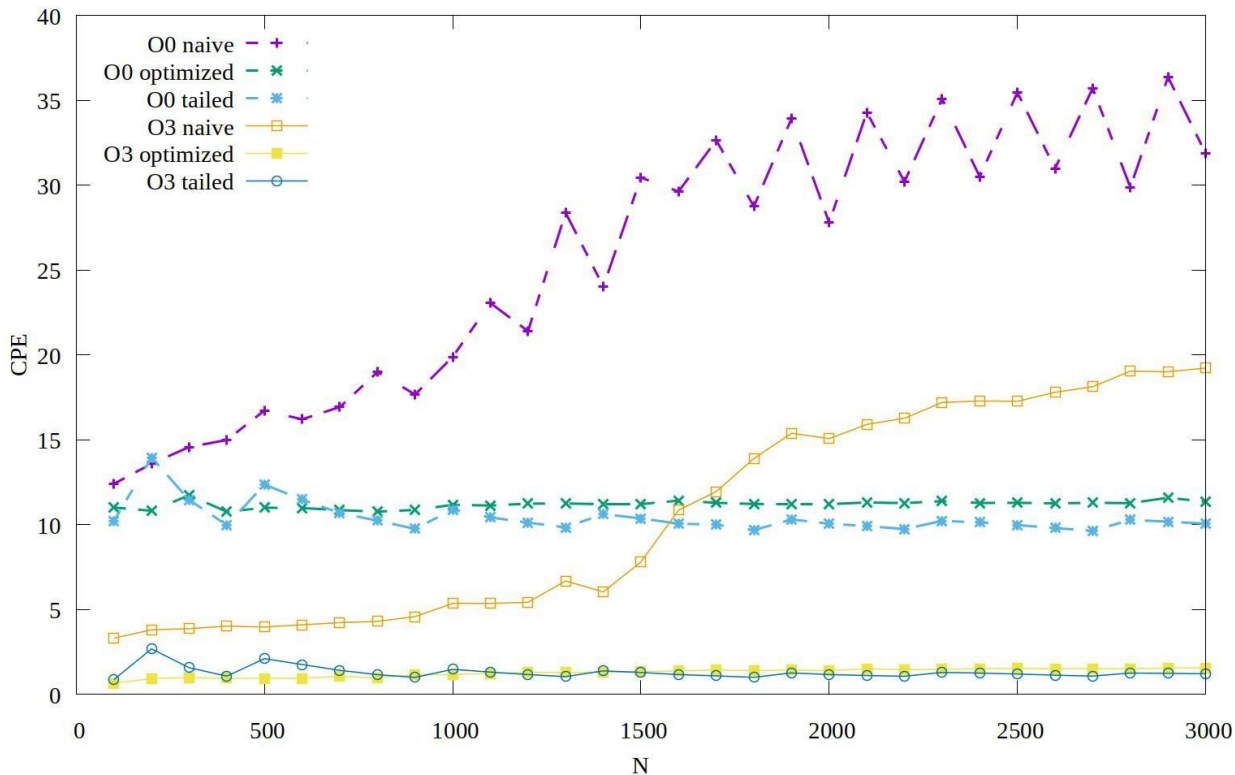
Istruzioni per ciclo

(più grande è, meglio è)



Loop

Risultati della moltiplicazione di matrici



Cicli per elemento

(più piccolo è, meglio è)